

Infosys Springboard Virtual Internship 6.0

Completion Report

Team Details

Batch Number- 11

Start date- 29/12/2025

Names:

Vishaal S

Soundaryan A

Utkarsh Deshmukh

Vinitha Y

Internship Duration: 8 Weeks

1. Project Title

Smart Inventory Management System

2. Project Objective

The main objective of this internship project was to design and develop a Smart Inventory Management System that enables businesses to efficiently track, manage, and analyze their inventory using a secure and centralized digital platform. Traditional inventory management systems are often manual, reactive, and prone to errors, resulting in operational inefficiencies, financial losses, and poor decision-making.

The project aimed to address critical challenges such as stockouts, overstocking, lack of demand forecasting, and delayed reporting. Inventory imbalances frequently lead to lost sales opportunities and increased storage costs, negatively impacting overall business profitability. Additionally, the absence of intelligent insights and predictive mechanisms prevents organizations from anticipating demand fluctuations and optimizing their inventory levels.

To solve these issues, the system was designed to provide real-time inventory tracking, automated low-stock alerts, secure role-based access control, and detailed transaction reporting. The platform enables administrators and employees to manage products, monitor stock movements, and generate exportable reports, improving operational visibility and accuracy.

The relevance of this project to the organization lies in its ability to enhance inventory efficiency, reduce human errors, improve decision-making speed, and support scalable digital transformation. By implementing modern technologies such as REST APIs, secure authentication, and real-time dashboards, the system helps organizations maintain optimal stock levels, prevent losses, and improve overall business performance.

3. Project description in detail

The Smart Inventory Management System is a full-stack, enterprise-grade web application designed to modernize and automate inventory management processes for businesses. The primary purpose of this system is to provide a centralized, secure, and intelligent platform that enables organizations to efficiently manage inventory, track stock movements, monitor product availability, and generate analytical reports. Traditional inventory systems often rely on manual record-keeping or fragmented tools, which lead to errors, delayed updates, lack of visibility, and inefficient decision-making. This project addresses these limitations by implementing a real-time, secure, and scalable inventory management solution.

The system follows a modern client-server architecture consisting of a React-based frontend, a Spring Boot backend, and a MongoDB database. The frontend was developed using React and Vite to provide a responsive and interactive user interface. Tailwind CSS and Material UI were used for styling and component design, ensuring a clean, professional, and user-friendly experience. The frontend communicates with the backend through RESTful APIs using HTTP requests, allowing seamless data exchange and real-time updates.

The backend was implemented using Spring Boot, which provides a robust and scalable framework for building enterprise applications. Spring Security was integrated to implement secure authentication and authorization mechanisms. JWT (JSON Web Token) authentication was implemented to enable stateless session management, ensuring secure communication between client and server. Each request made to the backend is validated using JWT a token, preventing unauthorized access and protecting sensitive data. Passwords are securely stored using BCrypt hashing, ensuring that user credentials remain protected even in the event of a data breach.

The system implements Role-Based Access Control (RBAC), which ensures that users can only access features and data relevant to their roles. Two primary roles were defined: Administrator and Employee. Administrators have full system access, including managing users, configuring product thresholds, viewing reports, and monitoring system activity. Employees have limited access, allowing them to view products, record transactions, and respond to alerts. This hierarchical access model enhances system security and ensures proper access control.

The product management module is a core component of the system, allowing administrators to manage inventory items efficiently. Users can perform Create, Read, Update, and Delete (CRUD) operations on products. Each product contains detailed attributes such as product name, SKU (Stock Keeping Unit), supplier information, category, price, current quantity, and minimum stock threshold. The system ensures data consistency by validating inputs and preventing duplicate entries. This module provides businesses with complete control over their inventory catalog.

The transaction management module allows users to record stock movements, including purchases (stock-in) and sales (stock-out). Each transaction is securely processed and stored in the database with details such as product ID, transaction type, quantity, timestamp, and user information. When a transaction is performed, the backend verifies user authentication and permissions, updates the product quantity atomically in the database, and logs the transaction for audit purposes. This ensures accurate and reliable tracking of inventory movements and maintains a complete transaction history.

One of the key features of the system is the automated low-stock alert mechanism. The system continuously monitors inventory levels and compares them against predefined minimum thresholds. When a product's quantity falls below the threshold, the system automatically generates an alert and displays it on the dashboard. This allows administrators to take timely action, such as restocking products, thereby preventing stockouts and operational disruptions. This feature significantly improves inventory reliability and business continuity.

The reporting and analytics module provides users with the ability to generate detailed inventory reports. These reports include product details, stock levels, transaction history, and low-stock alerts. Users can export reports in PDF and CSV formats, making them suitable for audits, business analysis, and record-keeping. The reporting system improves transparency and helps organizations make informed decisions based on accurate data.

The system also incorporates a machine learning-based demand forecasting module as an advanced feature. This module uses Python-based microservices and data analysis libraries such as Pandas and NumPy to analyze historical transaction data. Predictive models such as ARIMA and Prophet are used to forecast future inventory demand for a period ranging from 7 to 90 days. The forecasting pipeline includes data collection, preprocessing, feature engineering, model training, and prediction generation. This enables businesses to anticipate future demand, optimize inventory levels, and reduce losses caused by overstocking or stockouts.

The system architecture ensures scalability, modularity, and maintainability. The frontend, backend, database, and machine learning components are loosely coupled, allowing independent development and deployment. REST APIs provide a standardized communication mechanism between components. MongoDB was used as the database due to its flexibility, scalability, and ability to handle structured and semi-structured data efficiently.

Security was a major focus during development. Multiple security measures were implemented, including JWT authentication, BCrypt password hashing, role-based authorization, input validation, and CORS protection. Axios interceptors were used on the frontend to automatically attach authentication tokens to requests and handle session expiration. These measures ensure that the system remains secure and resistant to common vulnerabilities such as unauthorized access and injection attacks.

The system workflow begins when a user logs into the application using secure authentication. Once authenticated, users can view and manage products, record transactions, and monitor alerts. When a transaction is performed, the backend validates the request, updates the inventory database, and triggers alerts if necessary. Administrators can monitor system activity through dashboards and generate reports for analysis. This workflow ensures real-time synchronization between user actions and database updates.

Overall, the Smart Inventory Management System provides a comprehensive and intelligent solution for inventory management. It improves operational efficiency, enhances security, enables real-time monitoring, and supports data-driven decision-making. The system reduces manual effort, minimizes errors, prevents inventory shortages, and provides valuable business insights through reporting and forecasting. This project demonstrates the effective integration of modern web technologies, secure backend systems, scalable databases, and machine learning techniques to solve real-world business problems.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Studied project requirements and designed system architecture. Started implementing user authentication using Spring Boot and JWT.	Successfully set up the project environment, backend structure, and implemented basic user login and registration functionality.
Week 2	Implement Role-Based Access Control (RBAC) and secure authentication system. Configure Spring Security and JWT token validation.	Completed JWT authentication, role-based authorization, and secure login system for Admin and Employee roles.
Week 3	Design and implement product management module including product schema and database integration using MongoDB.	Developed product CRUD APIs and integrated MongoDB database. Enabled adding, viewing, updating, and deleting products.
Week 4	Implement stock level management and product search functionality. Ensure proper tracking of inventory quantities.	Completed stock management logic, product listing, search functionality, and frontend integration with backend APIs.
Week 5	Implement a low-stock alert system and threshold configuration. Design alert triggering mechanism.	Successfully implemented automated low-stock alerts, threshold monitoring, and alert display on the dashboard.
Week 6	Develop a transaction management module for recording stock-in and stock-out operations.	Implemented transaction APIs and frontend components to record purchases and sales with real-time stock updates.
Week 7	Integrate transaction history tracking and ensure accurate inventory updates and audit logging.	Completed transaction history module, audit logs, and ensured accurate and consistent stock updates.
Week 8	Implement reporting module and export functionality. Generate PDF and CSV reports. Perform testing and finalize the system.	Successfully implemented inventory reporting, PDF/CSV export functionality, system testing, debugging, and final project completion.

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Initiated the Smart Inventory Management System project by analyzing requirements, defining project scope, selecting technology stack (React, Spring Boot, MongoDB, Python for ML), and designing the overall system architecture including frontend, backend, database, and ML integration plan.	13/01/2026
Prototype / First Draft	Developed the initial working prototype including JWT-based authentication, role-based access control, and frontend and backend integration. Implemented secure login, user roles, and basic product management with database connectivity.	24/01/2026
Mid-Term Review	Completed core system modules including product management, stock tracking, and low-stock alert system. Integrated the machine learning demand forecasting module using Python, Pandas, and NumPy to analyze historical transaction data and generate future demand predictions. Demonstrated system functionality including real-time updates, alert triggering, and ML-based predictive insights.	07/02/2026
Final Submission	Successfully implemented all modules including transaction management, automated alerts, reporting system with PDF/CSV export, and ML forecasting integration. Completed system testing, debugging, performance optimization, and ensured secure and reliable system operation.	13/02/2026
Presentation	Prepared and delivered a comprehensive project presentation explaining system architecture, workflow, frontend and backend integration, security implementation, database design, and machine learning forecasting module. Demonstrated the working application and explained its real-world business impact.	28/02/2026

5b. Project execution details

1. Project Planning and Requirement Analysis

The execution of the Smart Inventory Management System began with a comprehensive planning and requirement analysis phase. The primary objective was to design and develop a scalable, secure, and intelligent inventory management platform capable of addressing real-world business challenges such as inefficient inventory tracking, stockouts, overstocking, and delayed reporting.

During this phase, detailed research was conducted to understand traditional inventory management systems and their limitations. It was observed that many small and medium-scale businesses rely on manual processes or fragmented software solutions that lack automation, security, and predictive capabilities. These limitations often result in data inconsistencies, delayed decision-making, and operational inefficiencies.

Based on the analysis, the core functional requirements of the system were defined, including secure user authentication, role-based access control, product management, transaction tracking, low-stock alert generation, and report generation. In addition to these core features, advanced capabilities such as machine learning-based demand forecasting were planned to enhance the system's intelligence and predictive capabilities.

The technology stack was carefully selected to ensure scalability, performance, and maintainability. React and Vite were chosen for frontend development due to their high performance and component-based architecture. Spring Boot and Spring Security were selected for backend development to provide robust REST API support and enterprise-grade security. MongoDB was chosen as the database due to its flexibility and ability to handle dynamic inventory data efficiently. Python, along with libraries such as Pandas and NumPy, was selected for implementing machine learning-based demand forecasting.

A modular system architecture was designed to separate frontend, backend, database, and machine learning components. This ensured clear separation of concerns, improved maintainability, and allowed independent development and testing of each module.

2. System Architecture Design

The system was designed using a client-server architecture to ensure scalability and security. The architecture consists of four main components:

1. Frontend Layer
2. Backend Layer
3. Database Layer
4. Machine Learning Layer

The frontend layer was developed using React and provides the graphical user interface for users. It includes modules such as login, dashboard, product management, transaction management, alerts, and reports.

The backend layer was implemented using Spring Boot and provides RESTful APIs for handling business logic, authentication, authorization, and data processing. Spring Security was integrated to enforce secure authentication and role-based access control.

The database layer uses MongoDB to store user data, product details, transaction history, and alert information. MongoDB provides high performance and scalability, making it suitable for inventory systems.

The machine learning layer was implemented as a Python-based module responsible for analyzing historical transaction data and generating demand forecasts.

Communication between frontend and backend was implemented using HTTP requests, and data was exchanged in JSON format. JWT authentication was used to ensure secure communication between components.

3. Frontend Development and User Interface Implementation

The frontend of the system was developed using React, which provides a component-based architecture for building scalable and reusable user interface components. Vite was used as the build tool to ensure fast development and optimized performance.

The user interface was designed to be intuitive, responsive, and user-friendly. Multiple pages and components were developed, including:

- Login Page
- Dashboard
- Product Management Page
- Transaction Management Page
- Alerts Page
- Reports Page

React state management was used to manage application state efficiently. Axios was used to communicate with backend APIs and retrieve data dynamically.

The dashboard was designed to provide users with a real-time overview of inventory status, including product availability, low-stock alerts, and recent transactions.

Input validation was implemented on the frontend to ensure correct data entry and prevent invalid inputs. Axios interceptors were used to automatically attach authentication tokens to API requests, ensuring secure communication.

The frontend was also designed to support responsive layouts, allowing the application to function properly on different screen sizes.

4. Backend Development and API Implementation

The backend was developed using Spring Boot, which provides a powerful framework for building secure and scalable web applications. The backend was responsible for implementing business logic, handling user authentication, managing inventory data, and processing transactions.

RESTful APIs were developed to perform various operations, including:

- User authentication and login
- Product management (Create, Read, Update, Delete)
- Transaction recording
- Alert generation
- Report generation

Spring Security was integrated to enforce authentication and authorization. JWT authentication was implemented to enable stateless session management. When users log in, the backend generates a JWT token, which is used to authenticate subsequent requests.

The backend verifies the token for every request and ensures that users have the necessary permissions to perform the requested operation.

BCrypt hashing was used to securely store user passwords, ensuring that sensitive data remains protected.

5. Database Design and Integration

MongoDB was used as the primary database for storing system data. Multiple collections were created, including:

- Users Collection
- Products Collection
- Transactions Collection
- Alerts Collection

Each collection was designed to store relevant data efficiently. For example, the Products collection stores product name, SKU, supplier, quantity, price, and threshold values.

The Transactions collection stores transaction type, quantity, timestamp, and user information.

MongoDB was integrated with Spring Boot using MongoDB drivers, allowing the backend to perform database operations efficiently.

Database indexing was used to improve performance and enable faster data retrieval.

6. Product Management Module Implementation

The product management module was developed to allow administrators to manage inventory items efficiently.

Users can perform CRUD operations, including:

- Adding new products
- Viewing product list
- Updating product information
- Deleting products

Each product contains detailed attributes such as product name, SKU, supplier, price, quantity, and minimum threshold.

The backend validates all inputs before storing them in the database to ensure data consistency.

7. Transaction Management and Real-Time Inventory Updates

The transaction management module was implemented to track inventory movements.

Users can record:

- Stock-in transactions (purchases)
- Stock-out transactions (sales)

When a transaction is recorded, the backend performs the following steps:

1. Authenticates the user using JWT
2. Validates transaction data
3. Updates product quantity in database
4. Stores transaction record
5. Checks threshold conditions

This ensures real-time inventory tracking and accurate data management.

8. Low Stock Alert System Implementation

An automated alert system was implemented to notify users when inventory levels fall below threshold values.

The backend continuously monitors product quantities. When the quantity falls below the threshold, an alert is generated and stored in the database.

The frontend displays alerts on the dashboard, allowing administrators to take timely action.

This feature helps prevent stockouts and ensures uninterrupted business operations.

9. Reporting and Export Functionality

The reporting module was developed to generate detailed inventory reports.

Reports include:

- Product details
- Stock levels
- Transaction history
- Alert information

Users can export reports in PDF and CSV formats.

This feature helps businesses maintain records and perform data analysis.

10. Machine Learning Module Implementation

An advanced machine learning module was implemented to provide demand forecasting capabilities.

Historical transaction data was collected from the database and processed using Python libraries such as Pandas and NumPy.

The machine learning pipeline included:

- Data collection
- Data preprocessing
- Feature engineering
- Model training
- Prediction generation

Forecasting models such as ARIMA and Prophet were used to predict future demand.

This enables businesses to plan inventory efficiently and prevent shortages.

11. Security Implementation

Multiple security measures were implemented to protect system data.

These include:

- JWT Authentication
- Role-Based Access Control
- Password Hashing using BCrypt
- Input validation
- Secure API communication

These measures ensure system security and data protection.

12. Testing, Debugging, and System Validation

The system was thoroughly tested to ensure reliability and performance.

Testing included:

- Functional testing
- API testing
- Authentication testing
- Database testing

Bugs were identified and resolved to improve system stability.

13. Deployment and Final System Integration

After completing development and testing, all modules were integrated into a fully functional system.

The frontend, backend, database, and machine learning modules were connected and tested together.

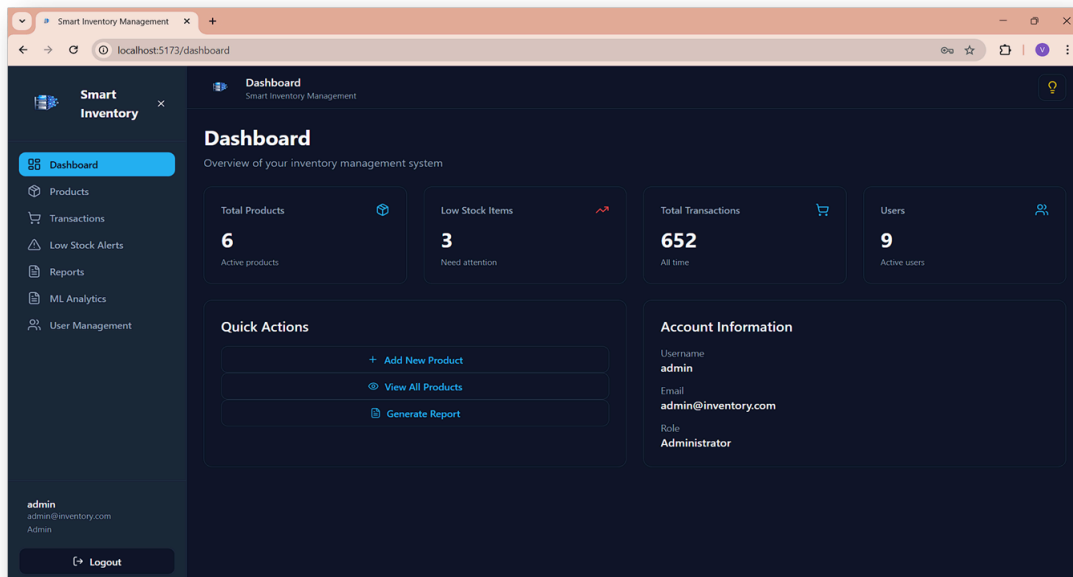
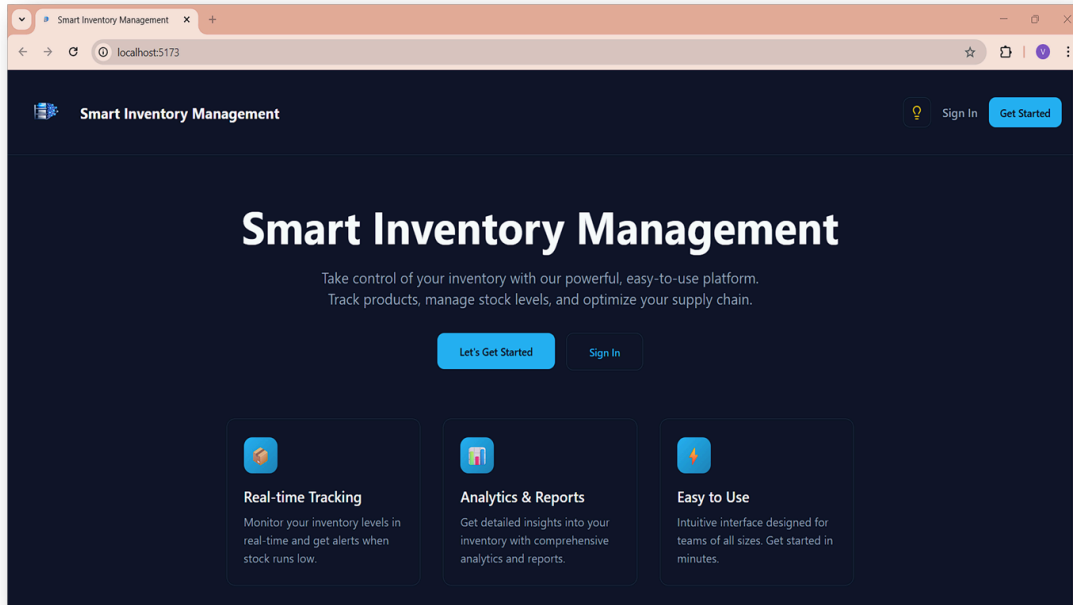
The system successfully performed all required operations, including authentication, product management, transaction tracking, alert generation, reporting, and demand forecasting.

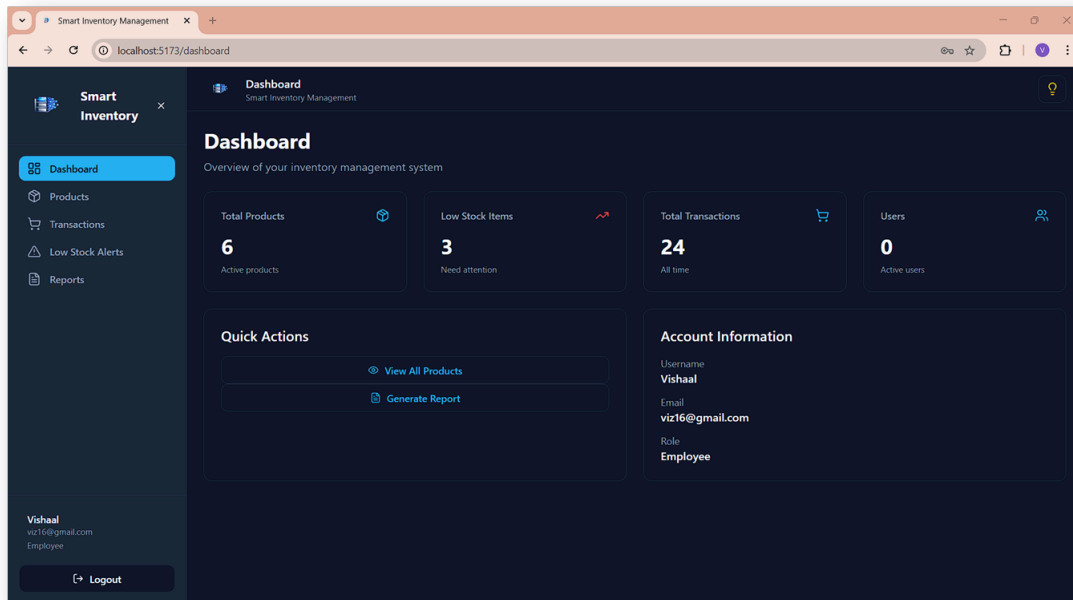
6. Snapshots / Screenshots

1. Backend structure:



2. Dashboard





3. Workflow



7. Challenges Faced

During the development and implementation of the Smart Inventory Management System, several technical and architectural challenges were encountered. These challenges provided valuable learning opportunities and helped improve problem-solving, debugging, and system design skills.

1. Implementing Secure JWT Authentication and Role-Based Access Control

One of the primary challenges was implementing a secure authentication system using JWT and integrating role-based access control (RBAC). Ensuring that only authorized users could access specific system features required careful configuration of Spring Security.

Initially, managing token generation, validation, and expiration handling was complex. There were issues with unauthorized access errors due to incorrect token parsing and missing authentication headers. Additionally, implementing role-based authorization required defining user roles and securing backend endpoints using appropriate security annotations.

This challenge was resolved by properly configuring Spring Security filters, implementing JWT token validation logic, and using role-based authorization annotations such as `@PreAuthorize`. Frontend Axios interceptors were also configured to automatically attach JWT tokens to API requests, ensuring secure communication between frontend and backend.

2. Frontend and Backend Integration Issues

Integrating the React frontend with the Spring Boot backend presented several challenges, particularly in managing API communication and handling asynchronous data updates. Issues such as incorrect API endpoints, CORS (Cross-Origin Resource Sharing) restrictions, and improper request handling initially prevented successful communication between frontend and backend.

These issues were resolved by configuring CORS policies in the backend, verifying API endpoint mappings, and using Axios for structured API communication. Debugging tools such as browser developer tools and backend logs were used to identify and resolve integration issues.

3. Database Schema Design and Data Consistency

Designing an efficient database schema for managing users, products, transactions, and alerts required careful planning. Ensuring data consistency during concurrent transactions was particularly challenging, as inventory quantities needed to be updated accurately.

There were initial issues with incorrect stock updates when multiple transactions were processed. This was resolved by implementing proper database update logic and ensuring atomic operations when updating inventory quantities. MongoDB queries were optimized to ensure reliable and efficient data storage and retrieval.

4. Real-Time Inventory Updates and Transaction Tracking

Implementing real-time inventory updates after each transaction required proper synchronization between frontend, backend, and database. Initially, there were delays in updating the frontend display after transactions were recorded.

This challenge was resolved by implementing proper state management in React and ensuring backend responses returned updated inventory data immediately. This allowed the frontend to refresh inventory data dynamically without requiring manual page reloads.

5. Implementing Low-Stock Alert System

Developing the automated low-stock alert system required implementing threshold-based monitoring logic. Ensuring that alerts were triggered accurately without generating duplicate or incorrect alerts was challenging.

This issue was resolved by implementing backend validation logic to check product quantities against defined threshold values after each transaction. Alerts were stored in the database and displayed on the frontend dashboard, ensuring accurate alert generation.

6. Machine Learning Model Integration and Data Processing

Integrating the machine learning demand forecasting module presented challenges related to data preprocessing, model training, and integration with the main system. Preparing historical transaction data for machine learning required cleaning missing values, formatting data properly, and ensuring sufficient data availability.

Additionally, connecting the Python-based machine learning module with the backend system required designing a structured workflow for data exchange and prediction generation.

This challenge was resolved by implementing proper data preprocessing using Pandas, organizing transaction data efficiently, and ensuring the machine learning module generated accurate predictions based on historical inventory data.

7. Ensuring System Security and Preventing Vulnerabilities

Ensuring the system was secure from unauthorized access and vulnerabilities required implementing multiple security measures. Improper input validation initially created potential security risks.

This was resolved by implementing strict input validation, secure password hashing using BCrypt, and enforcing authentication checks on all backend endpoints. JWT-based authentication ensured secure session management.

8. Debugging and Error Handling

Debugging errors across frontend, backend, and database layers was a continuous challenge. Errors such as incorrect API responses, database connection failures, and authentication errors required careful debugging.

This was resolved using logging tools, debugging techniques, and systematic testing of individual system components. Proper exception handling was also implemented to improve system reliability.

9. Managing System Complexity and Integration

Integrating multiple technologies including React, Spring Boot, MongoDB, and Python-based machine learning required careful coordination. Ensuring seamless communication between all system components was challenging.

This challenge was resolved by designing a modular architecture, testing each component independently, and gradually integrating all modules into the complete system.

10. Performance Optimization

Ensuring fast system performance and efficient data handling required optimizing database queries and frontend rendering. Initially, some operations were slow due to inefficient data handling.

This was resolved by optimizing database queries, improving API response handling, and implementing efficient frontend state management.

8. Learnings & Skills Acquired

The development of the Smart Inventory Management System provided extensive practical exposure to full-stack application development, secure backend implementation, database management, and machine learning integration. This internship significantly enhanced both technical expertise and professional skills.

1. Backend Development & Security

During this internship, I gained strong hands-on experience in building RESTful APIs using Spring Boot and implementing secure backend architectures. I developed authentication and authorization mechanisms using JWT and configured Spring Security to implement Role-Based Access Control (RBAC). I learned how stateless session management works through token validation mechanisms and ensured secure credential storage using BCrypt password encryption. Additionally, I implemented structured exception handling, input validation, and secured API endpoints to protect the system from unauthorized access and vulnerabilities.

2. Database Design & NoSQL Integration

I designed scalable and flexible database schemas using MongoDB, ensuring efficient storage and retrieval of inventory-related data. I implemented CRUD operations using repository patterns and learned indexing strategies to improve query performance. Special attention was given to maintaining data consistency by ensuring atomic stock updates during concurrent transactions. I

also efficiently managed transaction logs and alert collections to maintain accurate system records and audit trails.

3. Frontend Development

On the frontend, I developed interactive and responsive user interfaces using React. I built reusable components and managed application state effectively to ensure smooth user experience. The frontend was integrated with backend APIs using Axios, and JWT token handling was implemented through interceptors to maintain secure communication. I also designed dashboards that provide real-time monitoring of inventory status, transactions, and alerts.

4. Machine Learning & Data Analytics

I integrated a Python-based analytics module into the Java backend system to provide intelligent insights. Using Pandas and NumPy, I performed data preprocessing and transformation on historical transaction data. I implemented forecasting and optimization models such as Moving Average forecasting, ABC Analysis based on the Pareto principle, Reorder Point (ROP) calculation, Economic Order Quantity (EOQ), and anomaly detection using Isolation Forest. Through this, I gained a strong understanding of statistical concepts including service level, safety stock, standard deviation, and demand variability, and built a complete data pipeline from extraction to prediction integration.

5. System Design & Architecture

I designed a modular client-server architecture that separates frontend, backend, database, and analytics layers to ensure scalability and maintainability. I understood the importance of separation of concerns and implemented REST-based communication between services. This experience strengthened my ability to design structured, scalable, and enterprise-ready systems that integrate multiple technologies seamlessly.

9. Testimonials from the Team

During my internship on the Smart Inventory Management System, I received positive feedback from mentors and team members for my ability to independently design and implement complex modules such as JWT-based authentication, role-based access control using Spring Security, and the integration of a Python-based analytics engine with the Spring Boot backend. The team appreciated my structured approach to problem-solving, especially while handling database consistency in MongoDB, resolving integration issues between frontend and backend, and implementing advanced features like demand forecasting and anomaly detection. My contribution in building a responsive frontend using React and designing intelligent modules such as reorder point calculation and inventory analytics was recognized as adding significant value to the project.

The team also acknowledged my consistency in meeting weekly milestones, maintaining clean and well-documented code, and actively participating in discussions related to system architecture and optimization. Overall, the experience was highly collaborative and rewarding, and the successful completion of the project demonstrated my capability to handle full-stack development along with data-driven feature implementation in a real-world environment.

10. Conclusion

The internship experience of developing the Smart Inventory Management System was highly enriching and transformative. It provided an opportunity to apply academic knowledge to a real-world business problem and to design a complete, enterprise-level solution from concept to deployment. Through the implementation of secure backend services using Spring Boot and Spring Security, responsive frontend development using React, database management with MongoDB, and integration of advanced analytics using Python libraries such as Pandas, I was able to build a comprehensive and intelligent inventory platform.

This internship significantly enhanced my understanding of system architecture, secure authentication mechanisms, data management, and the practical application of machine learning in solving business challenges such as demand forecasting and inventory optimization. It strengthened my technical foundation, improved my problem-solving abilities, and exposed me to real-world software development practices including modular design, API integration, testing, debugging, and performance optimization.

Overall, this experience aligns strongly with my academic background in computer science and my career goal of becoming a full-stack developer with expertise in intelligent and data-driven systems. The knowledge and skills acquired during this internship have prepared me to contribute effectively to future projects and continue advancing in modern software development and applied machine learning domains.

11. Acknowledgements

I would like to express my sincere gratitude to Infosys for providing me with the valuable opportunity to undertake this internship and work on the Smart Inventory Management System. The experience has been highly enriching and instrumental in strengthening both my technical and professional skills.

I am especially thankful to my mentor for her continuous guidance, encouragement, and support throughout the project. Her insights, constructive feedback, and technical direction played a crucial role in helping me successfully design, develop, and complete the system.

I would also like to extend my appreciation to my team members for their collaboration, knowledge sharing, and teamwork during the internship period. Working together in a supportive and cooperative environment greatly enhanced my learning experience.

Finally, I am grateful for the overall learning atmosphere, resources, and opportunities provided during this internship, which have significantly contributed to my professional growth and confidence as a software developer.